Durante las entregas de los servicios de contactos, tareas y citas utilicé principalmente técnicas de pruebas unitarias y pruebas basadas en requisitos. Estas técnicas me permitieron verificar que cada método de los servicios funcionara de acuerdo con las condiciones establecidas y que los datos fueran manejados correctamente. Para realizar las pruebas utilicé JUnit, lo que permitió automatizar las verificaciones y repetirlas después de realizar cambios en el código. Las pruebas unitarias se enfocan en comprobar unidades individuales de un programa, como métodos o clases, de manera aislada (García, 2017).

En el servicio de contactos, las pruebas se enfocaron en operaciones como agregar, actualizar, eliminar y recuperar contactos. También se verificaron condiciones relacionadas con los datos de entrada, por ejemplo, que los identificadores fueran válidos y que los campos requeridos cumplieran con las restricciones establecidas. En el servicio de tareas utilicé un enfoque similar, comprobando que las tareas pudieran agregarse, actualizarse y eliminarse correctamente y que los métodos respondieran de manera apropiada cuando se proporcionaban datos inválidos. Para el servicio de citas, las pruebas permitieron verificar las operaciones correspondientes y las restricciones de los datos de las citas. En los tres servicios utilicé afirmaciones de JUnit, como assertEquals, assertNotNull y assertThrows, para comprobar resultados esperados y comportamientos relacionados con errores.

Otra técnica que utilicé fue la prueba de valores límite. Esta técnica consiste en comprobar los valores que se encuentran en los extremos de los rangos permitidos, ya que estos pueden revelar errores que no siempre aparecen con valores normales. Por ejemplo, si un campo tiene una longitud máxima determinada, es importante probar un valor que se encuentre exactamente en el límite y otro que lo exceda. También utilicé pruebas de casos negativos mediante excepciones, especialmente cuando un método debía rechazar información que no cumpliera con los

requisitos. Este enfoque permite comprobar no solamente que el programa funcione correctamente cuando recibe información válida, sino también que responda de forma controlada ante entradas incorrectas.

Existen otras técnicas de prueba que no utilicé directamente en estas entregas. Una de ellas son las pruebas de integración, cuyo propósito es verificar que diferentes componentes funcionen correctamente cuando interactúan entre sí. En un proyecto con una base de datos, por ejemplo, una prueba de integración podría comprobar que un servicio pueda guardar y recuperar información correctamente desde la base de datos. Otra técnica son las pruebas de sistema, que evalúan la aplicación completa desde una perspectiva más cercana al comportamiento que experimentaría el usuario. Estas pruebas pueden incluir diferentes componentes y verificar que el sistema cumpla con los requisitos funcionales establecidos.

También existen las pruebas de rendimiento, que permiten evaluar aspectos como el tiempo de respuesta, el consumo de recursos y el comportamiento de una aplicación cuando recibe una cantidad elevada de solicitudes. Estas pruebas serían especialmente importantes en sistemas que deben atender muchos usuarios simultáneamente. De manera similar, las pruebas de seguridad buscan identificar vulnerabilidades y problemas relacionados con la protección de los datos. En una aplicación financiera, por ejemplo, sería importante comprobar que los usuarios no puedan acceder a información que no tienen autorización para consultar.

Las técnicas de prueba tienen diferentes usos dependiendo del proyecto y de la etapa del desarrollo. Las pruebas unitarias son especialmente útiles durante la construcción del software porque permiten detectar errores en métodos y componentes individuales antes de que estos problemas afecten otras partes de la aplicación. Las pruebas de integración son más importantes cuando varios componentes comienzan a trabajar juntos, mientras que las pruebas de sistema

ayudan a verificar el comportamiento general de la aplicación. Por otra parte, las pruebas de rendimiento y seguridad adquieren mayor importancia cuando el software será utilizado en un entorno real y debe manejar información sensible, muchos usuarios o requisitos específicos de disponibilidad.

En conclusión, las entregas de contactos, tareas y citas me permitieron comprender que las pruebas de software no consisten solamente en verificar si un programa produce el resultado esperado. También es necesario comprobar cómo responde ante datos incorrectos, límites y situaciones inesperadas. El uso de JUnit y de pruebas unitarias me permitió identificar errores de manera temprana y comprobar que los servicios cumplieran con sus requisitos. Al mismo tiempo, conocer otras técnicas, como las pruebas de integración, sistema, rendimiento y seguridad, me permite comprender que la estrategia de pruebas debe adaptarse a las características y riesgos de cada proyecto.

Referencias

García, B. (2017). *Mastering software testing with JUnit 5*. Packt Publishing.

Myers, G. J., Sandler, C., & Badgett, T. (2011). *The art of software testing* (3rd ed.). Wiley.

Pressman, R. S., & Maxim, B. R. (2020). *Software engineering: A practitioner's approach* (9th ed.). McGraw-Hill Education.